

category: camera-integration

tags: [yt-dlp, youtube, opencv, cv2, video-stream, python]

severity: medium

created: 2026-07-02

updated: 2026-07-02 (thêm: seek/tua cho YouTube VOD + chọn độ phân giải)

project-origin: KZTEK Image Tools (d:\Tool)

Nhận dạng YOLO liên tục từ link YouTube: dùng yt-dlp resolve URL trước khi đưa vào cv2.VideoCapture

Tình huống gặp phải

Tab YOLO Detect ([tool/features/detection/tab_yolo.py](#)) đã có nhận dạng liên tục từ file video / RTSP / HTTP stream / webcam qua `cv2.VideoCapture(source)`. Yêu cầu thêm nguồn "link YouTube" — người dùng dán URL [youtube.com/watch?v=...](#), tool tự chạy YOLO detect liên tục trên video đó giống hệt các nguồn khác.

Triệu chứng / Lỗi

`cv2.VideoCapture("https://www.youtube.com/watch?v=...")` **không mở được** — OpenCV/ffmpeg không tự parse trang HTML YouTube để lấy stream thật, `cap.isOpened()` trả `False`.

Nguyên nhân gốc rễ (Root Cause)

URL YouTube dạng `watch?v=` chỉ là trang web, không phải link media trực tiếp. Cần một bước resolve riêng để lấy URL stream thật (progressive mp4 hoặc HLS manifest) rồi mới đưa vào `cv2.VideoCapture`.

Giải pháp

Dùng thư viện `yt-dlp` (không dùng `pytube/pafy` — cả hai đã lỗi thời, hay bị YouTube chặn) để extract info mà **không tải file** (`skip_download: True`), lấy field `url` trong info dict — đó chính là direct stream URL mà `cv2.VideoCapture` mở trực tiếp qua ffmpeg backend.

```
import yt_dlp

def _resolve_youtube_stream(url: str) -> tuple:
    ydl_opts = {
        "quiet": True,
        "no_warnings": True,
        "noplaylist": True,
        "skip_download": True,
        # progressive mp4 (video+audio 1 stream) trước; fallback "best" cho live/HLS
    }
```

```

    "format": "best[ext=mp4][protocol^=http]/best[protocol^=http]/best",
}
with yt_dlp.YoutubeDL(ydl_opts) as ydl:
    info = ydl.extract_info(url, download=False)
    if "entries" in info and info["entries"]:
        info = info["entries"][0] # link playlist → lấy video đầu
    stream_url = info["url"]
    title = info.get("title") or url
    return stream_url, title

cap = cv2.VideoCapture(stream_url) # mở được ngay, đọc frame như video thường

```

1. `pip install yt-dlp` (không phải `pytube` — bị outdated/broken thường xuyên với YouTube)
2. Gọi `extract_info(url, download=False)` để lấy dict info, không tải file về đĩa
3. Ưu tiên format `best[ext=mp4][protocol^=http]` — progressive stream (video+audio chung 1 URL HTTP) mở trực tiếp bằng `cv2` không cần `ffmpeg` xử lý thêm gì đặc biệt
4. Fallback `best[protocol^=http]` rồi `best` — với live stream, `yt-dlp` có thể trả về HLS (.m3u8) manifest URL; `cv2` (build có `ffmpeg`) vẫn mở được HLS trực tiếp
5. **Chạy resolve trong thread nền** — `extract_info` mất 1-3 giây (network call), gọi trực tiếp trên main thread sẽ đứng UI Tkinter. Dùng `threading.Thread + root.after(0, callback)` để cập nhật UI an toàn sau khi có kết quả (giống pattern queue/poll đã dùng cho worker detect video khác trong file này)

Áp dụng lại (How to reuse)

- Khi thấy yêu cầu "detect/stream từ link YouTube/Facebook/TikTok..." → nghĩ ngay đến `yt-dlp extract_info(download=False)` lấy `info["url"]`, không tự viết parser HTML
- URL trả về từ `yt-dlp` có **thời hạn** (thường vài giờ) — không cache lâu dài, resolve lại mỗi lần mở
- Nếu cần cả audio riêng (không chỉ video), phải chọn format khác và có thể cần `ffmpeg` mux 2 stream — không áp dụng nếu chỉ cần frame ảnh cho YOLO (bỏ qua audio hoàn toàn là đủ)

Mở rộng: chọn độ phân giải + tua (seek) như file video

- format string `yt-dlp` nhận `[height<=N]` để giới hạn độ phân giải:
`f"best[height<=720][ext=mp4][protocol^=http]/best[height<=720][protocol^=http]/best[height<=720]/best"`
 — map cứng nhãn hiển thị (Best/1080p/.../Worst) → format string, không build chuỗi động từ input tự do của user (tránh format string injection lạ vào `yt-dlp`)
- **Seek hoạt động được** trên stream URL đã resolve (progressive mp4 hỗ trợ HTTP range requests) giống hệt file cục bộ: `cap.get(cv2.CAP_PROP_FRAME_COUNT)` trả số frame hợp lệ, `cap.set(cv2.CAP_PROP_POS_FRAMES, n)` seek đúng vị trí — đã verify bằng test thực tế (video 15fps/284 frames, seek tới frame 100 → đọc được frame 101 ngay sau đó)
- Seek **không** áp dụng được cho **livestream đang phát** (HLS chưa có tổng thời lượng cố định) — phải lấy `info.get("is_live")` hoặc `info.get("live_status") == "is_live"` từ `yt-dlp`

để tắt seek bar khi là live, chỉ bật khi là VOD

- Tái dùng UI seek bar sẵn có (vốn chỉ hiện với file cục bộ qua điều kiện `os.path.isfile(source)`) bằng cách thêm tham số `enable_seek: bool` cho hàm mở cửa sổ video, rồi OR vào điều kiện hiện seek bar — không viết lại logic seek riêng cho YouTube

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Đừng gọi `yt_dlp.YoutubeDL(...).extract_info()` trên main thread của Tkinter — dialog sẽ đứng hình vài giây khi mạng chậm
- ⚠ Nếu dialog bị đóng (Hủy / X) trong lúc thread đang resolve, callback `root.after(0, ...)` vẫn chạy sau đó và có thể touch widget đã destroy → phải có cờ `_dlg_open` guard trước khi update widget trong callback
- ⚠ Link playlist (`&list=...`) trả về `info["entries"]` thay vì field `url` trực tiếp — phải check `"entries" in info` và lấy phần tử đầu
- ⚠ Không dùng `pytube`/`pafy` cho project mới — cả hai thường xuyên hỏng khi YouTube đổi cấu trúc trang, `yt-dlp` được maintain tích cực hơn nhiều

Tham chiếu

- Project liên quan: KZTEK Image Tools (`d:\Tool`) — `tool/features/detection/tab_yolo.py`, hàm `_resolve_youtube_stream`, `dialog_open_video_detect`